

FÖRDJUPNINGSUPPGIFT · DATA SCIENCE MED AI-FOKUS

Polars i Python

Grunderna i Polars, jämförelse med Pandas, och en praktisk tillämpning på elförbrukningsdata

ipj

EC Utbildning – Data Science, YH

2026

1. Valt område

Jag har valt att fördjupa mig i **Polars**, ett DataFrame-bibliotek för Python. Under utbildningen har vi mest jobbat med Pandas, men Polars har dykt upp gång på gång i artiklar och diskussioner om databehandling – alltid med ett löfte om att vara betydligt snabbare. Jag ville testa det själv istället för att bara ta det för givet, och se vad skillnaden faktiskt innebär rent praktiskt, både i hur man skriver kod och i hur snabbt den kör.

2. Fokus

Fokus i uppgiften har varit att lära mig grunderna i Polars: hur man läser in data, filtrerar den, sorterar den och gör olika sammanställningar. Jag har också velat förstå *varför* Polars ofta beskrivs som snabbare än Pandas – inte bara att det är det, utan vad som faktiskt ligger bakom skillnaden rent tekniskt (till exempel hur data lagras i minnet, och hur biblioteken använder processorns kärnor).

Jag har gått igenom följande delar i den praktiska notebooken:

- Inläsning av data och en första utforskning (schema, saknade värden, statistik)
- Filtrering av rader utifrån villkor
- Sortering på en och flera kolumner
- Beräkningar av nya kolumner, inklusive villkorsbaserade beräkningar
- Gruppering och aggregering (summor, medelvärden) på en eller flera kolumner
- Lazy evaluation – Polars sätt att bygga och optimera en hel körplan innan den exekveras
- En direkt jämförelse mellan Polars och Pandas, både i syntax och i hastighet

3. Praktisk del

Den praktiska delen av uppgiften finns i en separat Jupyter-notebook (`polars_fordjupning.ipynb`), där all kod är körd och kommenterad. Kortfattat gjorde jag så här:

1. **Skapade en exempeldatamängd.** Eftersom jag har en bakgrund inom kraftnätsbranschen (bland annat från E.ON och Eltel) byggde jag ett syntetiskt dataset med daglig elförbrukning för fem svenska städer (Stockholm, Göteborg, Malmö, Uppsala, Umeå), uppdelat på sektorerna hushåll, industri och kommersiell verksamhet, under hela 2025. Datasetet innehåller 5 475 rader och har en inbyggd säsongsvariation (högre förbrukning vintertid) samt en del saknade värden i temperaturkolumnen, för att kunna öva på att hantera sådant.
2. **Läste in och utforskade datan** med `pl.read_csv()`, kontrollerade schema och saknade värden.
3. **Filtrerade** fram delmängder, till exempel dagar med hög förbrukning i en specifik stad, och rader utan saknade temperaturvärden.
4. **Sorterade** datan på förbrukning, både enkelt och på flera kolumner samtidigt.
5. **Räknade ut nya kolumner**, till exempel total kostnad (förbrukning × pris) och en kategorisering av dagar som "Kall", "Mild" eller "Varm" utifrån temperatur, med `pl.when().then()`.
6. **Grupperade och summerade** förbrukning per region, per region och sektor, samt per månad – vilket bland annat bekräftade att min syntetiska data faktiskt hade den säsongsvariation jag byggt in.
7. **Testade lazy evaluation** med `pl.scan_csv()` och jämförde med den vanliga, "eager" varianten.
8. **Gjorde en prestandajämförelse** mot Pandas på en större testfil med en miljon rader, se avsnitt 5.

Exempel på kod

Så här ser en typisk gruppering ut i Polars, jämfört med motsvarande i Pandas:

```
# Polars
resultat = (
    df.group_by("region")
    .agg(pl.col("forbrukning_kwh").sum().alias("total_forbrukning_kwh"))
    .sort("total_forbrukning_kwh", descending=True)
)

# Pandas
resultat = (
```

```
df.groupby("region")["forbrukning_kwh"]
    .sum()
    .reset_index(name="total_forbrukning_kwh")
    .sort_values("total_forbrukning_kwh", ascending=False)
)
```

Resultatet blir detsamma, men jag tycker Polars-versionen är lite tydligare att läsa – man ser direkt vilken kolumn som summeras och vad den nya kolumnen ska heta, i ett och samma uttryck. I Pandas behövs oftast några extra steg (`reset_index` , namnge om kolumnen) för att komma till samma resultat.

4. Resultat: Polars vs Pandas i praktiken

För att jämföra prestanda genererade jag en separat testfil med en miljon rader slumpad numerisk data, och mätte tiden det tog att läsa in filen, filtrera bort en del rader och sedan gruppera och beräkna medelvärden. Samma operationer kördes i både Pandas och Polars (både i vanligt och lazy-läge). Resultatet från min körning:

Metod	Tid	Jämfört med Pandas
Pandas	0,219 s	–
Polars (eager)	0,094 s	~2,3× snabbare
Polars (lazy)	0,087 s	~2,5× snabbare

Redan på en miljon rader var Polars mer än dubbelt så snabbt som Pandas för samma kedja av operationer (inläsning, filtrering och gruppering), och lazy-läget var ytterligare något snabbare än det vanliga. Skillnaden hade sannolikt blivit ännu större på en riktigt stor fil, eller på en maskin med fler CPU-kärnor, eftersom Polars parallelliserar arbetet automatiskt medan Pandas i grunden kör på en kärna.

5. Relevans

Polars är ett verktyg som används direkt inom Data Science-fältet för att bearbeta och analysera data, särskilt när datamängderna växer sig stora. Under utbildningen har vi mest jobbat med mindre dataset där Pandas fungerar utmärkt, men i verkliga projekt – till exempel loggdata, sensordata eller, som i mitt exempel, mätvärden från elnätet – kan datamängderna bli betydligt större. Att förstå både Pandas och Polars, och veta när det ena kan vara ett bättre val än det andra, känns som en relevant och praktisk kunskap att ta med sig in i branschen, oavsett om jag i framtiden jobbar mer mot data engineering eller mot analys.

6. Avgränsning

Jag har medvetet hållit mig till grundläggande funktioner i Polars: inläsning, filtrering, sortering, enkla beräkningar, gruppering/aggregering samt en introduktion till lazy evaluation. Jag har alltså inte gått in på mer avancerade delar som till exempel streaming av data som inte får plats i minnet, egna plugins/UDF:er skrivna i Rust, eller integration mot databaser och molntjänster. Jag har inte heller byggt något större system runt Polars – uppgiften är avgränsad till en fristående notebook som visar och förklarar de grundläggande funktionerna, vilket kändes rimligt för uppgiftens omfattning.

7. Slutsats och reflektion

Efter att ha jobbat igenom det här har jag fått en mycket bättre känsla för Polars, både i hur det skrivs och varför det ofta beskrivs som snabbare än Pandas. Syntaxen med `pl.col(...)` gör att man alltid vet exakt vilken kolumn ett uttryck gäller, och kombinationen av `with_columns` , `group_by` och `agg` känns logisk och lättläst när man väl vant sig vid den. Prestandaskillnaden märktes tydligt redan på en miljon rader i mitt test, vilket gör mig nyfiken på hur det skulle se ut på ännu större datamängder.

Jag tror inte att Pandas försvinner – det är fortfarande extremt utbrett och har ett enormt ekosystem av bibliotek byggda ovanpå det. Men för egna projekt där jag jobbar med större datamängder, eller där prestanda spelar roll, kommer jag definitivt att fortsätta använda Polars framöver. Om jag hade haft mer tid hade jag gärna testat Polars mot en riktig, öppen datamängd (till exempel elförbrukningsdata från Energimyndigheten) och tittat närmare på hur Polars hanterar filer som är för stora för att få plats i minnet.

8. Startkällor

-
- Polars officiella dokumentation: docs.pola.rs
 - Polars GitHub-repository: github.com/pola-rs/polars
 - Pandas officiella dokumentation: pandas.pydata.org/docs

Se den bifogade Jupyter-notebooken (polars_fordjupning.ipynb) för fullständig, körd kod med förklaringar.